

User Guide for `ducha.sty`

Template Maintainer
template@example.com

April 19, 2026

Abstract

This document explains how to use the custom package `ducha.sty`. It shows how to load the package, choose build options, configure version and release metadata, call the helper commands, use the bundled theorem environments, and work with the included table, figure, algorithm, and bibliography support.

Keywords: custom package, usage guide, LaTeX template

Document Metadata

Version: Conference Submission

License: [CC BY 4.0](#)

Copyright: Copyright 2026 Template Maintainer

Code: [Example repository](#)

Data: [Example dataset](#)

Project: [Project homepage](#)

Preprint: [Example arXiv page](#)

DOI: [10.0000/placeholder-doi](#)

Note: Replace these placeholder values with the metadata relevant to your release.

Contents

1	Quick Start	2
2	Package Options	2
3	Metadata and Release Profiles	3
4	Helper Commands	3
4.1	Document-Level Helpers	3
4.2	Todo Commands	4
4.3	Domain-Specific Math Helpers	4
5	Theorems and Cross-References	4
6	Figures, Tables, and Algorithms	5
6.1	External Images	5
6.2	TikZ and Width Control	5
6.3	Algorithm Example	6
7	Bibliography Workflow	6
8	Usage Checklist	6

1 Quick Start

Load `ducha.sty` immediately after the document class. The custom package already configures the page layout, theorem environments, cross-references, figures, tables, algorithms, todo notes, and bibliography support, so a minimal starting point looks like this:

```
\documentclass[a4paper]{article}
\usepackage[submission,biblatex]{ducha}
\duchasetup{
  version=journal,
  license={CC BY 4.0},
  code={Repository},
  code url={https://example.com/code}
}

\title{My Document}
\author{Author Name}
\date{\today}

\begin{document}
\maketitle
\printduchametadata
\templatelistoftodos
\tableofcontents
...
\printtemplatebibliography
\end{document}
```

Section 2 explains the package options, Section 3 covers version profiles and release metadata, Section 4 shows the helper commands, Section 5 illustrates the theorem and reference setup, and Section 6 shows the figure, table, and algorithm support. For broader LaTeX references, see [1, 2].

2 Package Options

The package is controlled by two independent package-option groups: one for review visibility and one for bibliography mode. Rich document metadata such as version, license, copyright, code, and data availability is configured separately with `\duchasetup{...}`.

Option	Effect	Typical use
<code>submission</code>	Hides todo notes and the todo list	Final or shareable PDF
<code>draft</code>	Shows todo notes and enables the todo list	Internal review build
<code>biblatex</code>	Uses Biber through the package helper	Default workflow
<code>natbib</code>	Uses BibTeX through the package helper	BibTeX-only submission flow

Table 1: Core options provided by `ducha.sty`

Use case	Goal	Recommended option	
		Visibility	Bibliography
Working draft	Keep review notes visible	<code>draft</code>	<code>biblatex</code>
	Match a BibTeX-only workflow	<code>draft</code>	<code>natbib</code>
Final output	Hide review notes	<code>submission</code>	<code>biblatex</code>
	Hide notes and use BibTeX	<code>submission</code>	<code>natbib</code>

Table 2: Example option combinations

In practice, most users will start with `\usepackage[submission,biblatex]{ducha}`, switch to `draft` only when they want to surface inline review annotations, and then describe the release itself through `\duchasetup{...}`.

3 Metadata and Release Profiles

Use `\duchasetup{...}` to declare release metadata without rewriting the package. This is where you specify whether the file is a full version, draft version, conference submission, journal submission, arXiv version, working paper, or some other release profile, along with license, copyright, code, and data information.

```
\duchasetup{
  version=conference,
  license={CC BY 4.0},
  license url={https://creativecommons.org/licenses/by/4.0/},
  copyright={Copyright 2026 Author Name},
  code={Project repository},
  code url={https://github.com/user/project},
  data={Companion dataset},
  data url={https://example.com/data},
  project={Project page},
  project url={https://example.com/project},
  preprint={arXiv page},
  preprint url={https://arxiv.org/abs/1234.56789},
  doi={10.0000/example},
  note={Optional internal or public release note.}
}
```

The current guide file is configured as **Conference Submission**, which is why the metadata block near the front matter shows a conference-oriented release profile.

Version value	Default printed label
full	Full Version
draft	Draft Version
conference	Conference Submission
journal	Journal Submission
arxiv	arXiv Version
preprint	Preprint
working-paper	Working Paper
tech-report	Technical Report
manuscript	Manuscript

Table 3: Built-in release profiles supported by `\duchasetup`

4 Helper Commands

4.1 Document-Level Helpers

The package exposes several helpers that matter in most documents:

```
\duchasetup{...}
\duchaversionlabel
\email{person@example.com}
\printduchametadata
\templatelistoftodos
\printtemplatebibliography
```

Key	Purpose
<code>version / version label</code>	Select or override the release label
<code>license / license url</code>	Describe the reuse license
<code>copyright</code>	Record ownership or rights text
<code>code / code url</code>	Link the software or repository
<code>data / data url</code>	Link the dataset or archive
<code>project / project url</code>	Link a homepage or landing page
<code>preprint / preprint url</code>	Link a public preprint page
<code>doi</code>	Print a clickable DOI
<code>note</code>	Add a short release-specific note

Table 4: Metadata keys exposed by `\duchasetup`

- `\duchasetup{...}` stores release metadata such as version, license, code, and data links.
- `\duchaversionlabel` expands to the current human-readable release label.
- `\email{...}` creates a clickable mailto link and is useful in the author block.
- `\printduchametadata` prints the metadata block for the fields you configured.
- `\templatelistoftodos` prints the todo list only in **draft** mode.
- `\printtemplatebibliography` chooses the bibliography command automatically. It works with both `biblatex` and `natbib`.

4.2 Todo Commands

Todo helpers are available in every document, but they only render visibly when the package is loaded with the **draft** option:

```
\Task{Rewrite the introduction}
\TaskPerson{Editor}{Check the notation section}
\TaskDone{Bibliography verified}
\TaskNewPerson{Editor}{Editor}{blue}
\Editor{Rewrite the abstract}
\TaskEditor[highpriority]{Update Figure 2}
```

This design lets one source file serve both as a collaborative draft and as a clean submission build. In **submission** mode, these commands are effectively silent.

4.3 Domain-Specific Math Helpers

The package also bundles a set of graph-reconfiguration macros from the original project. If your article is in that area, they are ready to use:

$$\mathrm{TS}_k(G), \quad \mathrm{TJ}_k(G), \quad \langle I_0, \dots, I_t \rangle, \quad I \overset{\mathrm{TS}}{\leftrightarrow} J, \quad \mathrm{dist}(u, v), \quad \mathrm{CP}_n.$$

If your document is unrelated to this notation, you can ignore those commands without affecting the rest of the package.

5 Theorems and Cross-References

The package predefines common theorem-style environments such as **theorem**, **lemma**, **corollary**, **proposition**, **definition**, **example**, **remark**, and **problem**. It also loads `cleveref`, so references are printed automatically with the correct labels.

Definition 5.1. A *configured document* is a $\mathrm{L}^{\mathrm{A}}\mathrm{T}_{\mathrm{E}}\mathrm{X}$ file that loads `ducha.sty` and uses its helper commands instead of duplicating preamble setup locally.

Theorem 5.2. *If a configured document is compiled with the `submission` option, then review annotations created by the `todo` helpers do not appear in the output.*

Proof. The package wraps the `todo` commands and the `todo` list helper in conditional checks on the `submission` flag. When that flag is true, the visible note-producing branches are skipped. $\square$

Example 5.3. The sentence “Definition 5.1 and Theorem 5.2 describe the document setup and its clean-output behavior” is generated with ordinary `\cref` commands; the package handles the labels automatically.

Remark 5.4. If you need a new theorem-like environment that is not already bundled, add it in the package file so every document using the package receives the same numbering and reference behavior.

6 Figures, Tables, and Algorithms

The package already loads `graphicx`, `booktabs`, `multirow`, `tikz`, `adjustbox`, and `algorithm2e`. Keep figure assets in `figs/`; the package sets `\graphicspath{{figs/}}` for you.

6.1 External Images

Typical image commands are available immediately:

```
\includegraphics[width=0.45\textwidth]{example}
\includegraphics[height=3cm,keepaspectratio]{example}
\includegraphics[width=0.45\textwidth,trim=1cm 1cm 1cm 1cm,clip]{example}
```

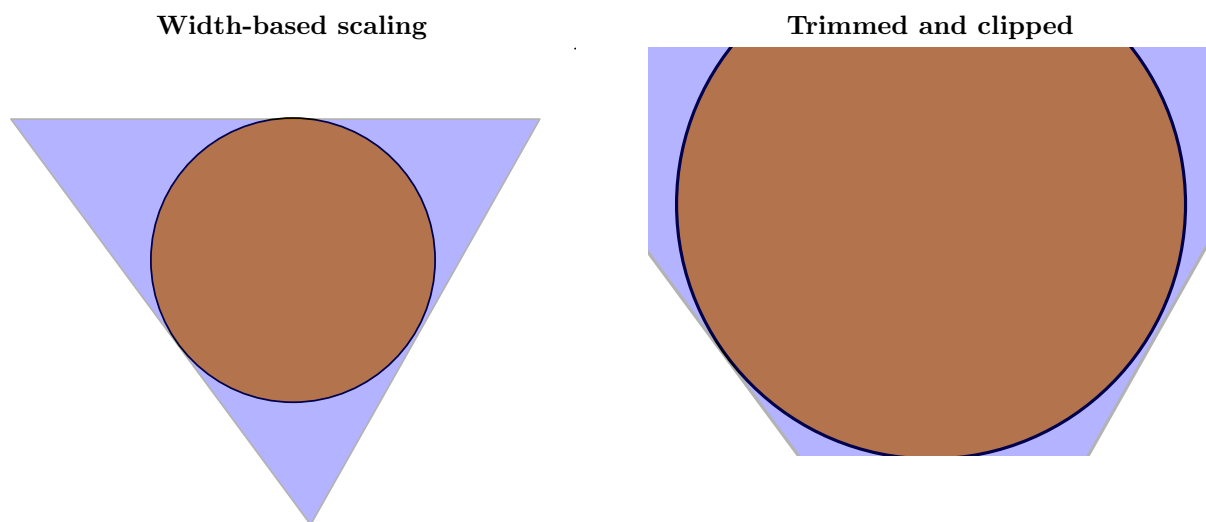


Figure 1: Two common `\includegraphics` patterns using `figs/example.pdf`

6.2 TikZ and Width Control

The template wraps TikZ figures in `adjustbox` so they scale down automatically when they exceed the text width.

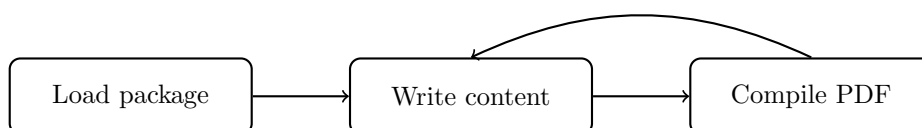


Figure 2: A simple package-usage workflow drawn directly in the source

6.3 Algorithm Example

The algorithm environment is ready for methods, procedures, or reproducibility notes:

Algorithm 1: Choosing a build configuration

Input: A document state, a release target, and a bibliography preference

Output: A package option pair plus metadata settings

- 1 Choose `draft` if review notes should remain visible; otherwise choose `submission`;
 - 2 Choose `biblatex` for the default Biber workflow; otherwise choose `natbib`;
 - 3 Set `\duchasetup` keys for version, license, copyright, code, and data as needed;
 - 4 Load `ducha` with the resulting option pair;
 - 5 Print the metadata block if you want that information visible in the front matter;
 - 6 Compile the document and place the bibliography with `\printtemplatebibliography`;
-

7 Bibliography Workflow

Store bibliography data in `refs.bib`. The package defaults to `biblatex` with Biber, but the same source file can switch to `natbib` with BibTeX by changing the package option:

```
\usepackage[submission,biblatex]{ducha}  
\usepackage[submission,natbib]{ducha}
```

Use ordinary citation commands in the body, for example [\[1, 2\]](#). Finish the document with `\printtemplatebibliography` so the package can select the correct bibliography command automatically.

8 Usage Checklist

When you start a new document with this package, the practical checklist is short:

1. Load `ducha` with the right visibility and bibliography options.
2. Use `\duchasetup{...}` to describe the release profile, license, copyright, code, and data information.
3. Keep figure assets in `figs/` and bibliography data in `refs.bib`.
4. Place `\printduchametadata` near the front matter if that metadata should appear in the PDF.
5. Place `\templatelistoftodos` near the front matter if you use `draft` mode.
6. Use the bundled theorem, figure, table, and algorithm environments instead of rebuilding the preamble.
7. End the document with `\printtemplatebibliography`.

References

- [1] Frank Mittelbach, Michel Goossens, Johannes Braams, David Carlisle, and Chris Rowley. *The LaTeX Companion*. 2nd ed. Addison-Wesley, 2004.
- [2] Leslie Lamport. *LaTeX: A Document Preparation System*. 2nd ed. Addison-Wesley, 1994.